

What did the company want to understand?

So the company didn't really know what exactly should be analysed and the point of job was to give a view of financial health in general of the company on the material that he could handover to me.

Why does this matter for their business?

It does actually matter, simply because the company needs to have an idea of what's going on and what could be changed in order to make the company grow not only financially but also physically by hiring people, taking more contracts, purchasing more equipment ect...

What limitations existed?

The very first issue I've encountered was not the fact that all what I've got is the purchasing and payments received but that whole information was spread out across 82 columns in a thousand of rows in (XLSX worksheet's) and in order to feed the power bi app the file should be in a structured preferred CSV file to further be able move the data, restructure and be more flexible with data in general

Project Background

Objective:

The client did not have a predefined analytical scope. Instead, the engagement was open-ended: review the available financial data and produce a general assessment of the company's financial health, using whatever raw material could be handed over.

Business Relevance:

Despite the lack of a specific ask, this analysis was highly valuable to the business. A clear view of financial performance gives leadership the insight needed to make informed decisions around growth — whether that means hiring new staff, pursuing additional contracts, or investing in equipment. In short, understanding the numbers is a prerequisite for scaling the business, both financially and operationally.

Constraints & Challenges:

The primary limitation was the state of the source data:

- The only data available consisted of purchasing records and payments received.

- This information was spread across **82 columns and roughly 3,000 rows** in an unstructured Excel (XLSX) worksheet.
- Before any analysis could begin, the data needed to be converted into a clean, structured **CSV format** suitable for ingestion into Power BI — enabling reliable restructuring, transformation, and flexible modelling downstream.

Data & Methodology:

- What data I had? (dates, payments, receipts)
- What data was not available? Useful for measuring project performance and profitability: Project ID, Project name, Project type, Location, Start date, Planned completion date, Actual completion date, Project status, Planned budget, Actual cost, Final profit or loss of project, Change orders, Retention amount, Completion percentage.

Questions this can answer:

- Which projects are profitable?
- Which types of projects generate the most revenue?
- Are projects completed on time?
- Which projects exceed their budgets?

Revenue and payments:

- Project ID
- Payment status
- Payment method
- Outstanding balance
- Possible analysis:
- Accounts receivable
- Late payments
- Average payment delay
- Revenue by project

Possible analysis:

- Late payments
- Average payment delay
- Revenue by client or project
- Cash-flow trends

Expenses and purchases:

- Purchase ID
- Project ID
- Material category
- Description
- Quantity
- Unit price
- Delivery cost

- Payment status

Possible analysis:

- Material costs by project
- Spending by supplier(in comparison)
- Cost trends over time
- Budget versus actual spending
- Most expensive materials
- Unexpected cost increases

Labor and employees

- Project ID
- Hours worked
- Overtime hours
- Hourly rate
- Labor cost
- Start and end dates

(There are usually many more questions in case of many employees like contractor or permanent employee status, absence or leave records, Employee ID etc)

Possible analysis:

- Labor cost by project
- Overtime trends
- Cost per project hour
- Productivity by project

- Whether labor costs are exceeding the budget

Materials and inventory:

- Material ID
- Material name
- Category
- Supplier
- Quantity purchased
- Quantity used
- Quantity remaining
- Unit of measurement
- Unit cost
- Reorder level
- Delivery date
- Waste or damaged quantity

Possible analysis:

- Material usage
- Waste percentage
- Stock levels
- Materials most frequently purchased
- Supplier price comparisons

Equipment and vehicles: important for understanding operational costs.

- Equipment ID
- Equipment type and all related information

- Maintenance cost
- Usage hours
- Project assigned
- Breakdown dates
- Downtime

Possible analysis:

- Equipment cost by project
- Owned versus rented equipment
- Maintenance trends
- Equipment utilization
- Downtime
- Cost of equipment breakdowns

Sales and quotations: useful for understanding business growth.

- Quote ID
- Project type
- Estimated value
- Estimated cost
- Quoted margin
- Lead source

Possible analysis:

- Quote win rate
- Most profitable project types
- Average quote value

- Difference between estimated and actual project profitability

Schedule and progress data:

- Task ID
- Project ID
- Task name
- Planned start date
- Planned end date
- Actual start date
- Actual end date
- Task status
- Percentage complete
- Delay days
- Reason for delay

Possible analysis:

- Planned versus actual completion
- Delayed tasks
- Average project delay
- Common causes of delay
- Relationship between delays and costs

Quality and defects:

- Inspection ID

- Project ID
- Inspection date
- Area or task inspected
- Defect type
- Severity
- Corrective action
- Rework cost
- Resolution date
- Inspection result

Possible analysis:

- Defect by project
- Rework costs
- Common quality issues
- Relationship between defects and project delay

1/3 Received entire data in XLSX format.

WPS Office

2023-ads-grootboek-boekplan.xlsx

2023-ads-grootboek-boekplan.xlsx

2023-ads-zamvordagboek.xlsx

Upgrade Now

FileEditViewInsertPage LayoutFormulasDataReviewViewToolsPremiumWPS AI

Not uploadedShare

Format Painter

Paste

Font

Arial Narrow

9.75

A+

A-

B

I

U

Align

Color

Background Color

Text Color

General

Format Conversion

Rows and Columns

Table Styles

Cell Styles

Fill

AutoSum

Sort

Filter

Freeze Panes

Find

Orientation

Wrap Text

Merge

General

Format Conversion

Rows and Columns

Table Styles

Cell Styles

Fill

AutoSum

Sort

Filter

Freeze Panes

Find

D2398

fx

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB	AC	AD	AE	AF	AG	AH	AI	AJ	AK		
21																																							
22																																							
23																																							
24																																							
25																																							
26																																							
27																																							
28																																							
29																																							
30																																							
31																																							
32																																							
33																																							
34																																							
35																																							
36																																							
37																																							
38																																							
39																																							
40																																							
41																																							
42																																							
43																																							
44																																							
45																																							
46																																							
47																																							
48																																							
49																																							
50																</																							

2/3 Ran a Python script to extract the required information from the invoices and organize it into a structured CSV file for further analysis in tools such as SSMS and Power BI.

```

import re
import os
import pandas as pd

file = pd.read_excel(
    r'C:\python\bouwfirma\bouwfirma_data\data\raw\bouwfirma\2023 ads-
aanverdagboek.xlsx',
    header=None
)

# Excel range E14:CY2676
data = file.iloc[13:2676, 4:104].copy()

data = data.fillna("").astype(str)

# Looser patterns
invoice_start_pattern =
re.compile(r'^\s*([FC])\s*AK\s*(\d+)\s*H\s*(\d+(?:\.\d+)?)\b', re.IGNORECASE)
account_pattern = re.compile(r'^\s*(\d{6})\s+(.*\S)\s*$')
date_pattern = re.compile(r'(\d{4}-\d{2}-\d{2})')

records = []
current_record = None

def to_number(value):
    try:
        return float(str(value).replace(",", "."))
    except ValueError:
        return None

for _, row in data.iterrows():
    row_text = " ".join(row.fillna("").astype(str).tolist())
    row_text = re.sub(r"\s+", " ", row_text).strip()

    if not row_text:

```

continue

Detect invoice header

```
m = invoice_start_pattern.match(row_text)
```

```
if m:
```

```
    prefix = m.group(1).upper()
```

```
    ak_num = m.group(2)
```

```
    h_num = f"{int(float(m.group(3))):02d}"
```

Find the first two dates anywhere in the row

```
dates = date_pattern.findall(row_text)
```

```
start_date = dates[0] if len(dates) > 0 else None
```

```
end_date = dates[1] if len(dates) > 1 else None
```

Remove the prefix "F AK ... H .." from the row

```
after_header = row_text[m.end():].strip()
```

Supplier is everything before the first date

```
supplier = after_header
```

```
if start_date:
```

```
    supplier = after_header.split(start_date)[0].strip()
```

```
current_record = {
```

```
    "Invoice Number": f"{prefix} AK {ak_num} H {h_num}",
```

```
    "Supplier": supplier,
```

```
    "Start_Date": start_date,
```

```
    "End_Date": end_date,
```

```
    "Full_Price": None,
```

```

    "BTW-receive": None,
    "BTW-to-Pay": None,
    "Product_price": None,
    "%_of_sum": None,
    "Other_expansies": None,
    "Private": None
}
records.append(current_record)
continue

```

Detect account/detail rows under current invoice

if current_record:

```

    match = account_pattern.match(row_text)

```

if match:

```

    rest = match.group(2).strip()

```

```

    label = rest.lower()

```

Extract percentages like 21%

```

    pct = re.search(r'(\d+%)', rest)

```

Extract decimal numbers only, so we ignore things like VF060290

```

    decimals = re.findall(r'\d+[.]\d+', rest)

```

```

    decimals = [to_number(x) for x in decimals if to_number(x) is not None]

```

if "leveranciers" in label:

Example: Leveranciers 0 48.67 VF060290 -> want 48.67

if decimals:

```

    current_record["Full_Price"] = decimals[-1]

```

elif "terug te vorderen btw" in label:

Example: 8.45 0 -> want 8.45

if decimals:

```

    current_record["BTW-receive"] = decimals[0]

```

elif "btw te betalen" in label:

if decimals:

```

    current_record["BTW-to-Pay"] = decimals[0]

```

```

elif "aankopen grondstoffen" in label:
    # Example: 40.22 0 VF060290 21% -> want 40.22 and 21%
    if decimals:
        current_record["Product_price"] = decimals[0]
    if pct:
        current_record["%_of_sum"] = pct.group(1)

elif "restaurantkosten" in label or "onderhoud & herstellingen rollend
materieel" in label:
    if decimals:
        current_record["Other_expansies"] = decimals[0]

elif "relatiegeschenken" in label or "r/c bedrijfsleider" in label:
    if decimals:
        current_record["Private"] = decimals[0]

df = pd.DataFrame(records)

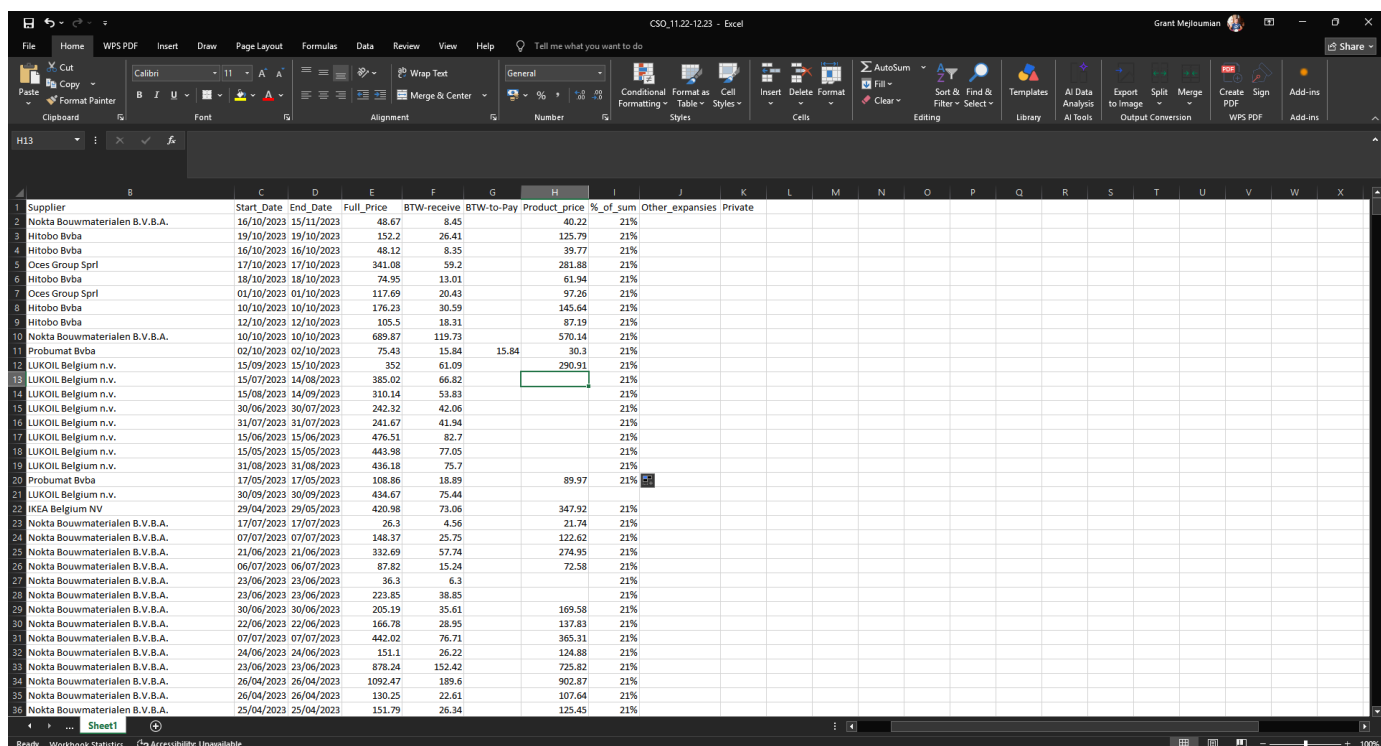
output_path =
r'C:\python\bouwfirma\bouwfirma_data\output\bouwfirma_structured_outpu
t.csv'
df.to_csv(output_path, index=False, encoding='utf-8-sig')

print(f"Saved to: {output_path}")
print(f"Rows written: {len(df)}")
print(df.head(10).to_string(index=False))
print(f"Current working directory: {os.getcwd()}")

```

3/3 After running the code on the source file, I noticed that many cells contained empty or NULL values. To investigate these gaps efficiently, I opened SQL Server Management Studio (SSMS) alongside my Python environment. I used the Python script to

identify the exact invoices containing missing information, then reviewed those invoices and manually entered the missing values into the appropriate cells in the SSMS table. This approach was more efficient than spending additional time modifying the extraction script to capture every piece of information from the inconsistently formatted invoices.



	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
1	Supplier	Start_Date	End_Date	Full_Price	BTW-receive	BTW-to-Pay	Product_price	%_of_sum	Other_expenses	Private													
2	Nokta Bouwmaterialen B.V.B.A.	16/10/2023	15/11/2023	48.67	8.45		40.22	21%															
3	Hitobo Bvba	19/10/2023	19/10/2023	152.2	26.41		125.79	21%															
4	Hitobo Bvba	16/10/2023	16/10/2023	48.12	8.35		39.77	21%															
5	Oces Group Sprl	17/10/2023	17/10/2023	341.08	59.2		281.88	21%															
6	Hitobo Bvba	18/10/2023	18/10/2023	74.95	13.01		61.94	21%															
7	Oces Group Sprl	01/10/2023	01/10/2023	117.69	20.43		97.26	21%															
8	Hitobo Bvba	10/10/2023	10/10/2023	176.23	30.59		145.64	21%															
9	Hitobo Bvba	12/10/2023	12/10/2023	105.5	18.31		87.19	21%															
10	Nokta Bouwmaterialen B.V.B.A.	10/10/2023	10/10/2023	689.87	119.73		570.14	21%															
11	Probumat Bvba	02/10/2023	02/10/2023	75.43	15.84	15.84	30.3	21%															
12	LUKOIL Belgium n.v.	15/09/2023	15/10/2023	352	61.09		290.91	21%															
13	LUKOIL Belgium n.v.	15/07/2023	14/08/2023	385.02	66.82			21%															
14	LUKOIL Belgium n.v.	15/08/2023	14/09/2023	310.14	53.83			21%															
15	LUKOIL Belgium n.v.	30/06/2023	30/07/2023	242.32	42.06			21%															
16	LUKOIL Belgium n.v.	31/07/2023	31/07/2023	241.67	41.94			21%															
17	LUKOIL Belgium n.v.	15/06/2023	15/06/2023	476.51	82.7			21%															
18	LUKOIL Belgium n.v.	15/05/2023	15/05/2023	443.98	77.05			21%															
19	LUKOIL Belgium n.v.	31/08/2023	31/08/2023	436.18	75.7			21%															
20	Probumat Bvba	17/05/2023	17/05/2023	108.86	18.89		89.97	21%															
21	LUKOIL Belgium n.v.	30/09/2023	30/09/2023	434.67	75.44			21%															
22	IKEA Belgium NV	29/04/2023	29/05/2023	420.98	73.06		347.92	21%															
23	Nokta Bouwmaterialen B.V.B.A.	17/07/2023	17/07/2023	26.3	4.56		21.74	21%															
24	Nokta Bouwmaterialen B.V.B.A.	07/07/2023	07/07/2023	148.37	25.75		122.62	21%															
25	Nokta Bouwmaterialen B.V.B.A.	21/06/2023	21/06/2023	332.69	57.74		274.95	21%															
26	Nokta Bouwmaterialen B.V.B.A.	06/07/2023	06/07/2023	87.82	15.24		72.58	21%															
27	Nokta Bouwmaterialen B.V.B.A.	23/06/2023	23/06/2023	36.3	6.3			21%															
28	Nokta Bouwmaterialen B.V.B.A.	23/06/2023	23/06/2023	223.85	38.85			21%															
29	Nokta Bouwmaterialen B.V.B.A.	30/06/2023	30/06/2023	205.19	35.61		169.58	21%															
30	Nokta Bouwmaterialen B.V.B.A.	22/06/2023	22/06/2023	166.78	28.95		137.83	21%															
31	Nokta Bouwmaterialen B.V.B.A.	07/07/2023	07/07/2023	442.02	76.71		365.31	21%															
32	Nokta Bouwmaterialen B.V.B.A.	24/06/2023	24/06/2023	151.1	26.22		124.88	21%															
33	Nokta Bouwmaterialen B.V.B.A.	23/06/2023	23/06/2023	878.24	152.42		725.82	21%															
34	Nokta Bouwmaterialen B.V.B.A.	26/04/2023	26/04/2023	1092.47	189.6		902.87	21%															
35	Nokta Bouwmaterialen B.V.B.A.	26/04/2023	26/04/2023	130.25	22.61		107.64	21%															
36	Nokta Bouwmaterialen B.V.B.A.	25/04/2023	25/04/2023	151.79	26.34		125.45	21%															

I identified the rows with NULL values in the Full_Price and Product_Price columns and populated the missing values using T-SQL queries.

Left window (Python IDE):

```
1 import pandas as pd
2 import re
3
4 file = pd.read_excel(r"C:\python\bouwfirma_data\data\raw\bouwfirma\2023 ads-aanverdagboe
5 data = file.iloc[13:2676, 4:184].fillna("")
6
7 header = re.compile(pattern=r'^(?=[FC])$AK$S$D$S$H$S$D$', re.IGNORECASE)
8
9 targets = [
10     "F AK 172 H 04",
11     "De Roeck Nv",
12     "2022-11-25"
13 ]
14
15 block = []
16 capture = False
17
18 def matches_any(block_lines): 2+ usages
19     joined = " ".join(block_lines).lower()
20     return any(t.lower() in joined for t in targets)
21
22 for _, row in data.iterrows():
23     row_text = " ".join(str(x) for x in row.tolist())
```

Right window (SQL Server Enterprise Manager):

SQLQuery1.sql - DESKTOP-4MSKIG6\Construction (DESKTOP-4MSKIG6\Grant (59)) - Microsoft SQL Server Manag...

```
1 select *
2 from [C50_11.22-12.23]
3 where Full_Price is null and Product_price is null
4 order by Start_Date
```

Results:

Invoice_Number	Supplier	Start_Date	End_Date	Full_Price	BTW_receive	BTW_to_Pay	Product_price	of_sum	Other_ex
1	FAK 172 H 04	2022-11-25	2022-12-25	NULL	NULL	NULL	NULL	NULL	NULL
2	FAK 181 H 04	2023-02-06	2023-02-06	NULL	NULL	NULL	NULL	NULL	NULL
3	FAK 182 H 04	2023-02-13	2023-02-13	NULL	NULL	NULL	NULL	NULL	NULL
4	FAK 185 H 04	2023-02-14	2023-03-16	NULL	NULL	NULL	NULL	NULL	NULL
5	FAK 89 H 04	2023-03-25	2023-03-25	NULL	NULL	NULL	NULL	NULL	NULL
6	FAK 208 H 05	2023-03-28	2023-05-13	NULL	NULL	NULL	NULL	NULL	NULL
7	FAK 189 H 04	2023-04-20	2023-04-20	NULL	57.0999984741211	NULL	NULL	NULL	NULL
8	FAK 272 H 05	2023-05-06	2023-05-06	NULL	NULL	NULL	NULL	NULL	NULL
9	FAK 273 H 05	2023-06-02	2023-06-02	NULL	NULL	NULL	NULL	0%	NULL
10	FAK 72 H 04	2023-06-07	2023-07-07	NULL	NULL	NULL	NULL	NULL	NULL
11	FAK 73 H 04	2023-06-13	2023-07-13	NULL	NULL	NULL	NULL	NULL	NULL
12	FAK 88 H 04	2023-06-17	2023-07-17	NULL	NULL	NULL	NULL	NULL	NULL
13	FAK 201 H 04	2023-06-25	2023-06-18	NULL	NULL	NULL	NULL	NULL	NULL
14	FAK 92 H 04	2023-07-03	2023-08-02	NULL	NULL	NULL	NULL	NULL	NULL
15	FAK 139 H 04	2023-08-22	2023-08-22	NULL	NULL	NULL	NULL	NULL	NULL
16	FAK 142 H 04	2023-08-22	2023-08-22	NULL	NULL	NULL	NULL	NULL	NULL
17	FAK 170 H 04	2023-09-04	2023-10-04	NULL	NULL	NULL	NULL	NULL	NULL
18	FAK 94 H 04	2023-09-06	2023-09-06	NULL	NULL	NULL	NULL	NULL	NULL

Left window (Python IDE):

```
1 import pandas as pd
2 import re
3
4 file = pd.read_excel(r"C:\python\bouwfirma_data\data\raw\bouwfirma\2023 ads-aanverdagboe
5 data = file.iloc[13:2676, 4:184].fillna("")
6
7 header = re.compile(pattern=r'^(?=[FC])$AK$S$D$S$H$S$D$', re.IGNORECASE)
8
9 targets = [
10     "F AK 172 H 04",
11     "De Roeck Nv",
12     "2022-11-25"
13 ]
14
15 block = []
16 capture = False
17
18 def matches_any(block_lines): 2+ usages
19     joined = " ".join(block_lines).lower()
20     return any(t.lower() in joined for t in targets)
21
22 for _, row in data.iterrows():
23     row_text = " ".join(str(x) for x in row.tolist())
```

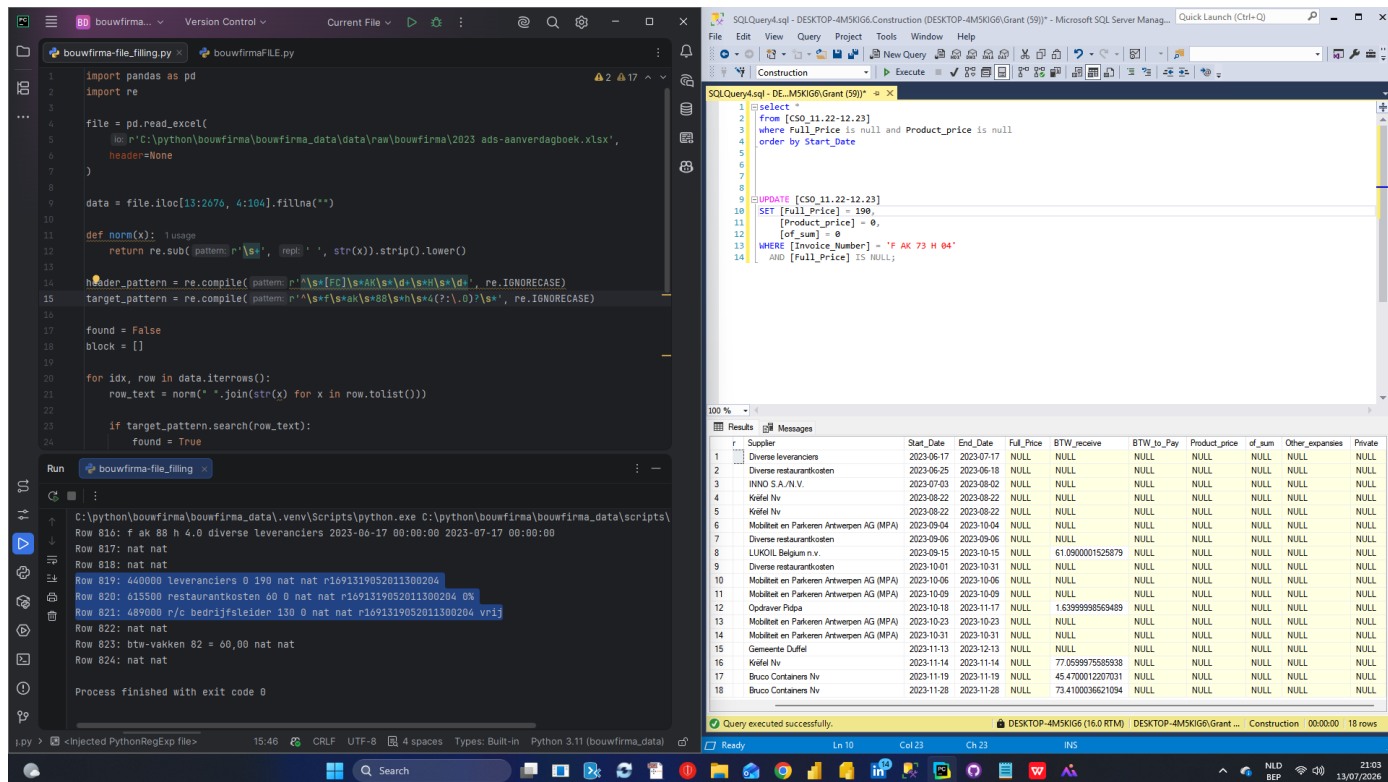
Right window (SQL Server Enterprise Manager):

SQLQuery1.sql - DESKTOP-4MSKIG6\Construction (DESKTOP-4MSKIG6\Grant (59)) - Microsoft SQL Server Manag...

```
1 select *
2 from [C50_11.22-12.23]
3 where Full_Price is null and Product_price is null
4 order by Start_Date
5
6
7
8
9
10 UPDATE [C50_11.22-12.23]
11 SET [Full_Price] = 456
12 WHERE [Invoice_Number] = 'F AK 172 H 04'
13 AND [Full_Price] IS NULL
```

Messages:

(1 row affected):
Completion time: 2026-07-19T19:13:00.4909012+02:00



However, I then realized it would be safer and faster to fill in the entire row with the correct information, rather than only updating the Full_Price column (VAT-inclusive) and using formulas to derive the Product_Price column (VAT-exclusive), which would then be used together with those two columns to calculate the %_of_sum column.

Python script for file filling:

```
import pandas as pd
import re

file = pd.read_excel(
    r"C:\python\bouwfirma\bouwfirma_data\data\bouwfirma\2023-ads-aanverdagboek.xlsx",
    header=None
)

data = file.iloc[13:2676, 4:104].fillna("")

def norm(x): 1 usage
return re.sub(pattern=r'[\s]', repl=' ', str(x)).strip().lower()

header_pattern = re.compile(pattern=r'[\s]{0,1}[\s]{0,1}[\s]{0,1}[\s]{0,1}', re.IGNORECASE)
target_pattern = re.compile(pattern=r'[\s]{0,1}[\s]{0,1}[\s]{0,1}[\s]{0,1}', re.IGNORECASE)

found = False
block = []

for idx, row in data.iterrows():
    row_text = norm(' '.join(str(x) for x in row.tolist()))

    if target_pattern.search(row_text):
        found = True
```

Run output:

```
Row 2430: f ak 269 h 5.0 bruce containers nv 2023-11-28 00:00:00 2023-11-28 00:00:00 462311131809.0
Row 2431: nat nat
Row 2432: nat nat
Row 2433: 440800 leveranciers 0 423 nat nat 23111318
Row 2434: 499800 terug te vorderen btw 73.41 0 nat nat 23111318
Row 2435: 612150 containerkosten 349.59 0 nat nat 23111318 21%
Row 2436: nat nat
Row 2437: btw-vakken 59 = 73.41 82 = 349.59 nat nat
Row 2438: nat nat
```

SQL Query:

```
select *
from [CSO_11.22-12.23]
where Full_Price is null and Product_price is null
order by Start_Date

UPDATE [CSO_11.22-12.23]
SET [Full_Price] = 423,
    [BTW_receive] = 73.41,
    [BTW_to_Pay] = 0,
    [Product_price] = 349.59,
    [of_sum] = 21,
    [Other_expenses] = 0,
    [Private] = 0
WHERE [Invoice_Number] = 'F AK 269 H 05'
AND [Full_Price] IS NULL;
```

Results:

Invoice_Number	Supplier	Start_Date	End_Date	Full_Price	BTW_receive	BTW_to_Pay	Product_price	of_sum	Other_expenses	Private
2430	f ak 269 h 5.0 bruce containers nv	2023-11-28	2023-11-28	00:00:00	00:00:00	462311131809.0				
2431	nat nat									
2432	nat nat									
2433	440800 leveranciers	0 423	nat nat	23111318						
2434	499800 terug te vorderen btw	73.41	0 nat nat	23111318						
2435	612150 containerkosten	349.59	0 nat nat	23111318	21%					
2436	nat nat									
2437	btw-vakken	59 = 73.41	82 = 349.59	nat nat						
2438	nat nat									

SQL Query:

```
select *
from [CSO_11.22-12.23]
where [Private] != 0

UPDATE [CSO_11.22-12.23]
SET [BTW_receive] = ROW([Full_price] * 0.21, 2)
WHERE [Supplier] IS NOT NULL
AND [Full_price] IS NOT NULL
AND [Invoice_Number] IN (
    'F AK 141 H 04',
    'F AK 143 H 04',
    'F AK 145 H 04',
    'F AK 256 H 05',
    'F AK 271 H 05',
    'F AK 274 H 05'
)
```

Results:

Invoice_Number	Supplier	Start_Date	End_Date	Full_Price	BTW_receive	BTW_to_Pay	Product_price	of_sum	Other_expenses	Private
1	F AK 89 H 04	Diverse brandstoffen	2023-03-25	2023-03-25	202	NULL	0	NULL	NULL	84.3
2	F AK 90 H 04	Parfuma Wijnegem	2023-09-16	2023-10-16	339.48	0	0	0	0	339.48
3	F AK 141 H 04	Ketel Nv	2023-08-22	2023-08-22	674.1	NULL	0	NULL	NULL	674.1
4	F AK 143 H 04	Ketel Nv	2023-08-22	2023-08-22	331.55	NULL	0	NULL	NULL	331.55
5	F AK 144 H 04	ALDI N.V.	2023-08-21	2023-09-20	59.99	NULL	0	NULL	NULL	59.99
6	F AK 145 H 04	Ketel Nv	2023-08-22	2023-08-22	569.05	62.49	62.49	297.55	21%	569.05
7	F AK 232 H 05	Studio de Dijk BV	2023-04-26	2023-05-26	2309.92	NULL	0	NULL	NULL	2309.92
8	F AK 256 H 05	Gamma - BG Retail N.V.	2023-11-30	2023-11-30	15.58	NULL	0	NULL	NULL	15.58
9	F AK 271 H 05	Brunatex	2023-10-12	2023-10-12	161.5	NULL	0	NULL	NULL	161.5
10	F AK 274 H 05	ACTION	2023-05-17	2023-05-17	24.24	NULL	0	NULL	NULL	24.24

```
bouwfirma-file_filling.py | bouwfirmaFILE.py
v data = file.iloc[13:2670, 4:104].fillna("")
10
11 def norm(x): fusage
12     return re.sub(pattern=r'[\s]', repl='', str(x)).strip().lower()
13
14 header_pattern = re.compile(pattern=r'^[S=]{F}[S=A][S=D][S=H][S=D$', re.IGNORECASE)
15 target_pattern = re.compile(pattern=r'^[S=f[S=A][S=2C4][S=5(?:\.)?][S=]', re.IGNORECASE)
16
17 found = False
18 block = []
19
20 for idx, row in data.iterrows():
21     row_text = norm(' '.join(str(x) for x in row.tolist()))
22
23     if target_pattern.search(row_text):
24         found = True
25         block = [f'Row {idx + 14}: {row_text}']
26         continue
27
28     if found:
29         if header_pattern.match(row_text):
30             break
31         if row_text:
32             block.append(f'Row {idx + 14}: {row_text}')
33
Run | bouwfirma-file_filling x
C:\python[bouwfirma]\bouwfirma_data\.venv\Scripts\python.exe C:\python[bouwfirma]\bouwfirma_data\scripts\
↓
Row 2206: f ak 246 h 5.0 sa axa belgium 2023-11-08 00:00:00 2023-11-08 00:00:00 624296680005.0
↑
Row 2207: nat nat
⌵
Row 2208: nat nat
⌴
Row 2209: 440800 Leveranciers 0 848.11 nat nat 06.12.2023-06.12.2024
⌶
Row 2210: 616120 verzekerling rollend materieel 848.11 0 nat nat 06.12.2023-06.12.2024_ba, 37988721_08
⌸
Row 2211: nat nat
⌹
Row 2213: btw-vakken 82 = 848.11 nat nat
⌺
Row 2214: nat nat
⌻
Process finished with exit code 0
```

The screenshot displays the Microsoft SQL Server Enterprise Manager interface. The main window shows a SQL query in the 'SQL Query Analyzer' pane. The query is as follows:

```

227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
11
```

```

1  # bouwfirma... Version Control v
2  # bouwfirmaFILE.py
3
4  )
5
6  data = file.iloc[13:2676, 4:104].fillna("")
7
8
9  def norm(x):
10     return re.sub( pattern: r'\s+', repl: ' ', str(x)).strip().lower()
11
12
13  header_pattern = re.compile( pattern: r'^\s*[FC]\s*[Aa][s][d][s][h]\s*[d]', re.IGNORECASE)
14  target_pattern = re.compile( pattern: r'^\s*[f][s][a][s][148][s][I][s]*2(?:\.)?\s*', re.IGNORECASE)
15
16  found = False
17  block = []
18
19  for idx, row in data.iterrows():
20     row_text = norm(' '.join(str(x) for x in row.tolist()))
21
22     if target_pattern.search(row_text):
23         found = True
24         block = [f"Row {idx + 14}: {row_text}"]
25         continue
26
27     if found:
28         if header_pattern.match(row_text):
29             break
30
31
32 Run
33
34 C:\python\bouwfirma\bouwfirma_data\venv\Scripts\python.exe C:\python\bouwfirma\bouwfirma_data\scripts\
35 Row 1376: f ak 148 1 2.0 www.tenpoplast.be 45462.0 45432.0 20992549009.0
36 Row 1379: 440000 leveranciers 0 1100 15512/2024
37 Row 1380: 411000 te ontvangen btw 231 0 15512/2024
38 Row 1381: 451000 te betalen btw 0 231 15512/2024
39 Row 1382: 600000 aankopen grondstoffen 1100 0 15512/2024 21%
40 Row 1384: btw-vakken 55 = 231.00 59 = 231.00 81 = 1.100,00 86 = 1.100,00
41
42 Process finished with exit code 0
43
44 j.py > C:\python\bouwfirma\bouwfirma_data\venv\Scripts\python.exe C:\python\bouwfirma\bouwfirma_data\scripts\
45 Row 1376: f ak 148 1 2.0 www.tenpoplast.be 45462.0 45432.0 20992549009.0
46 Row 1379: 440000 leveranciers 0 1100 15512/2024
47 Row 1380: 411000 te ontvangen btw 231 0 15512/2024
48 Row 1381: 451000 te betalen btw 0 231 15512/2024
49 Row 1382: 600000 aankopen grondstoffen 1100 0 15512/2024 21%
50 Row 1384: btw-vakken 55 = 231.00 59 = 231.00 81 = 1.100,00 86 = 1.100,00
51
52 Process finished with exit code 0
53
54 j.py > C:\python\bouwfirma\bouwfirma_data\venv\Scripts\python.exe C:\python\bouwfirma\bouwfirma_data\scripts\
55 Row 1376: f ak 148 1 2.0 www.tenpoplast.be 45462.0 45432.0 20992549009.0
56 Row 1379: 440000 leveranciers 0 1100 15512/2024
57 Row 1380: 411000 te ontvangen btw 231 0 15512/2024
58 Row 1381: 451000 te betalen btw 0 231 15512/2024
59 Row 1382: 600000 aankopen grondstoffen 1100 0 15512/2024 21%
60 Row 1384: btw-vakken 55 = 231.00 59 = 231.00 81 = 1.100,00 86 = 1.100,00
61
62 Process finished with exit code 0
63
64 j.py > C:\python\bouwfirma\bouwfirma_data\venv\Scripts\python.exe C:\python\bouwfirma\bouwfirma_data\scripts\
65 Row 1376: f ak 148 1 2.0 www.tenpoplast.be 45462.0 45432.0 20992549009.0
66 Row 1379: 440000 leveranciers 0 1100 15512/2024
67 Row 1380: 411000 te ontvangen btw 231 0 15512/2024
68 Row 1381: 451000 te betalen btw 0 231 15512/2024
69 Row 1382: 600000 aankopen grondstoffen 1100 0 15512/2024 21%
70 Row 1384: btw-vakken 55 = 231.00 59 = 231.00 81 = 1.100,00 86 = 1.100,00
71
72 Process finished with exit code 0
73
74 j.py > C:\python\bouwfirma\bouwfirma_data\venv\Scripts\python.exe C:\python\bouwfirma\bouwfirma_data\scripts\
75 Row 1376: f ak 148 1 2.0 www.tenpoplast.be 45462.0 45432.0 20992549009.0
76 Row 1379: 440000 leveranciers 0 1100 15512/2024
77 Row 1380: 411000 te ontvangen btw 231 0 15512/2024
78 Row 1381: 451000 te betalen btw 0 231 15512/2024
79 Row 1382: 600000 aankopen grondstoffen 1100 0 15512/2024 21%
80 Row 1384: btw-vakken 55 = 231.00 59 = 231.00 81 = 1.100,00 86 = 1.100,00
81
82 Process finished with exit code 0
83
84 j.py > C:\python\bouwfirma\bouwfirma_data\venv\Scripts\python.exe C:\python\bouwfirma\bouwfirma_data\scripts\
85 Row 1376: f ak 148 1 2.0 www.tenpoplast.be 45462.0 45432.0 20992549009.0
86 Row 1379: 440000 leveranciers 0 1100 15512/2024
87 Row 1380: 411000 te ontvangen btw 231 0 15512/2024
88 Row 1381: 451000 te betalen btw 0 231 15512/2024
89 Row 1382: 600000 aankopen grondstoffen 1100 0 15512/2024 21%
90 Row 1384: btw-vakken 55 = 231.00 59 = 231.00 81 = 1.100,00 86 = 1.100,00
91
92 Process finished with exit code 0
93
94 j.py > C:\python\bouwfirma\bouwfirma_data\venv\Scripts\python.exe C:\python\bouwfirma\bouwfirma_data\scripts\
95 Row 1376: f ak 148 1 2.0 www.tenpoplast.be 45462.0 45432.0 20992549009.0
96 Row 1379: 440000 leveranciers 0 1100 15512/2024
97 Row 1380: 411000 te ontvangen btw 231 0 15512/2024
98 Row 1381: 451000 te betalen btw 0 231 15512/2024
99 Row 1382: 600000 aankopen grondstoffen 1100 0 15512/2024 21%
100 Row 1384: btw-vakken 55 = 231.00 59 = 231.00 81 = 1.100,00 86 = 1.100,00
101
102 Process finished with exit code 0
103
104 j.py > C:\python\bouwfirma\bouwfirma_data\venv\Scripts\python.exe C:\python\bouwfirma\bouwfirma_data\scripts\
105 Row 1376: f ak 148 1 2.0 www.tenpoplast.be 45462.0 45432.0 20992549009.0
106 Row 1379: 440000 leveranciers 0 1100 15512/2024
107 Row 1380: 411000 te ontvangen btw 231 0 15512/2024
108 Row 1381: 451000 te betalen btw 0 231 15512/2024
109 Row 1382: 600000 aankopen grondstoffen 1100 0 15512/2024 21%
110 Row 1384: btw-vakken 55 = 231.00 59 = 231.00 81 = 1.100,00 86 = 1.100,00
111
112 Process finished with exit code 0
113
114 j.py > C:\python\bouwfirma\bouwfirma_data\venv\Scripts\python.exe C:\python\bouwfirma\bouwfirma_data\scripts\
115 Row 1376: f ak 148 1 2.0 www.tenpoplast.be 45462.0 45432.0 20992549009.0
116 Row 1379: 440000 leveranciers 0 1100 15512/2024
117 Row 1380: 411000 te ontvangen btw 231 0 15512/2024
118 Row 1381: 451000 te betalen btw 0 231 15512/2024
119 Row 1382: 600000 aankopen grondstoffen 1100 0 15512/2024 21%
120 Row 1384: btw-vakken 55 = 231.00 59 = 231.00 81 = 1.100,00 86 = 1.100,00
121
122 Process finished with exit code 0
123
124 j.py > C:\python\bouwfirma\bouwfirma_data\venv\Scripts\python.exe C:\python\bouwfirma\bouwfirma_data\scripts\
125 Row 1376: f ak 148 1 2.0 www.tenpoplast.be 45462.0 45432.0 20992549009.0
126 Row 1379: 440000 leveranciers 0 1100 15512/2024
127 Row 1380: 411000 te ontvangen btw 231 0 15512/2024
128 Row 1381: 451000 te betalen btw 0 231 15512/2024
129 Row 1382: 600000 aankopen grondstoffen 1100 0 15512/2024 21%
130 Row 1384: btw-vakken 55 = 231.00 59 = 231.00 81 = 1.100,00 86 = 1.100,00
131
132 Process finished with exit code 0
133
134 j.py > C:\python\bouwfirma\bouwfirma_data\venv\Scripts\python.exe C:\python\bouwfirma\bouwfirma_data\scripts\
135 Row 1376: f ak 148 1 2.0 www.tenpoplast.be 45462.0 45432.0 20992549009.0
136 Row 1379: 440000 leveranciers 0 1100 15512/2024
137 Row 1380: 411000 te ontvangen btw 231 0 15512/2024
138 Row 1381: 451000 te betalen
```

SQL Query6.sql - DESKTOP-4MSKIG6.Construction (DESKTOP-4MSKIG6.Grant (58)) - Microsoft SQL Server Manager...

File Edit View Query Project Tools Window Help

SQL Query6.sql - DESKTOP-4MSKIG6.Grant (58) * x [CSO_11:24:12.23] - MSKIG6.Grant (51)

```

26 use Construction;
27
28 --update[CSO_01.24-12.24]
29 set
30     [Full_Price] = 572,
31     [BTW_receive] = 99.27,
32     [BTW_to_Pay] = 0,
33     [Product_price] = 0,
34     [of_sum] = 21,
35     [Other_expenses] = 0,
36     [Private] = 0
37
38 where [Invoice_Number] = 'F AK 147 H 02'
39
40 --select distinct [supplier]
41 --from [CSO_01.24-12.24]
42 --where Product_price is null
43
44 --select
45 --from [CSO_01.24-12.24]
46 --where Full_Price is null
47
48 --insert into [CSO_01.24-12.24].[Invoice_Number,
49 --Supplier,
50
51

```

100 %

Results Messages

	Invoice_Number	Supplier	Start_Date	End_Date	Full_Price	BTW_receive	BTW_to_Pay	Product_price	of_sum	Other_expenses	Private
109	F AK 112 H 01	Europharm NV	2024-04-04	2024-04-04	71153	NULL	NULL	NULL	NULL	NULL	NULL
110	F AK 112 H 01	LUNOKI Belgium n.v.	2024-02-15	2024-02-15	47903	66.12	NULL	NULL	NULL	NULL	NULL
111	F AK 113 H 01	ACERTA SOCIAL SECRET...	2023-12-31	2023-12-31	25.92	4.5	NULL	NULL	NULL	NULL	NULL
112	F AK 114 H 01	ACERTA SOCIAL SECRET...	2024-01-02	2024-01-02	131.66	NULL	NULL	NULL	NULL	NULL	NULL
113	F AK 115 H 01	ACERTA SOCIAL SECRET...	2024-01-31	2024-01-31	26.05	4.52	NULL	NULL	NULL	NULL	NULL
114	F AK 115 H 01	Van Marcke N.V.	2024-02-07	2024-03-08	2209.68	383.5	NULL	NULL	NULL	NULL	NULL
115	F AK 127 H 01	Europharm NV	2024-03-04	2024-03-04	784.62	154.77	154.77	794.62	21%	NULL	NULL
116	F AK 120 H 01	ACERTA SOCIAL SECRET...	2024-02-29	2024-02-29	26.05	4.52	NULL	NULL	NULL	NULL	NULL
117	F AK 121 H 01	LUNOKI Belgium n.v.	2024-02-29	2024-02-29	441.38	66.43	NULL	NULL	NULL	NULL	NULL
118	F AK 122 H 01	Santo Depot	2024-03-01	2024-03-31	652.35	113.22	NULL	539.13	21%	NULL	NULL
119	F AK 123 H 01	LUNOKI Belgium n.v.	2024-03-15	2024-03-15	664.66	139.58	139.58	664.66	21%	NULL	NULL
120	F AK 128 H 01	COLORIUT	2024-03-23	2024-03-23	206.43	NULL	NULL	NULL	NULL	NULL	206.4
121	F AK 126 H 01	Hedbo Bvba	2024-03-29	2024-03-29	164.31	28.52	NULL	135.79	21%	NULL	NULL
122	F AK 127 H 01	Mobiliteit en Parkeren Antwer...	2024-03-14	2024-03-14	4	0	0	0	0	0	44
123	F AK 128 H 01	Diverse restaurantkosten	2024-03-24	2024-03-24	23.35	NULL	NULL	NULL	NULL	NULL	23.35
124	F AK 129 H 01	Diverse restaurantkosten	2024-03-23	2024-03-23	71	0	0	0	0	0	0
125	F AK 130 H 01	Hedbo Bvba	2024-03-25	2024-03-25	1336.63	231.98	NULL	1104.65	21%	NULL	NULL
126	F AK 131 H 01	HAA BVBA	2024-03-13	2024-03-13	38.99	6.77	NULL	32.22	21%	NULL	NULL

Query executed successfully.

DESKTOP-4MSKIG6 (16.0 RTM) DESKTOP-4MSKIG6.Grant... Construction 00:00:00 534 rows

Ready Ln115 Col1 IRS

12:05 22/07/2026

Using the same approach, I cleaned, completed, and consolidated the purchase files from **2022 through 2025**. I applied the same process to the client payment files, combining the data into a single structured dataset for further analysis in Power BI.